Requested Citation Acknowledgment: IEEE OTCBVS WS Series Bench; J. Davis and M. Keck, "A two-stage approach to person detection in thermal imagery," In Proc. Workshop on Applications of Computer Vision, January 2005

Imports

In [ ]:
```
### Object Detection with Faster R-CNN and RetinaNet in PyTorch ###
# !pip -q install torch torchvision torchmetrics matplotlib pandas pycocotools faster-coco-eval tqdm

%pip -q install torch torchvision torchmetrics matplotlib
%pip install pandas
%pip install pycocotools
%pip install faster-coco-eval
%pip install tqdm

import re, random, time
from pathlib import Path
from typing import List, Dict, Tuple

import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from tqdm.auto import tqdm

import torch
from torch.utils.data import Dataset, DataLoader

import torchvision
from torchvision.transforms import functional as TF
from torchvision.models.detection import (
    RetinaNet_ResNet50_FPN_V2_Weights, FasterRCNN_ResNet50_FPN_V2_Weights,
    retinanet_resnet50_fpn_v2, fasterrcnn_resnet50_fpn_v2,
)
from torchvision.models.detection.retinanet import RetinaNetClassificationHead
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchmetrics.detection.mean_ap import MeanAveragePrecision
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Data Preprocessing and Creating Data Loaders

In [ ]:
```
### Helper function for loading and parsing the OSU Thermal Pedestrian Database (OTCBVS) ground-truth files. ###
### Needs to be adapted if you want to use a different dataset. ###


# OSU Thermal Pedestrian Database (OTCBVS) ground-truth parser
# File format:
#   NumberOfFrames
#   img_00001.bmp 4 (x1 y1 x2 y2) (x1 y1 x2 y2) ...
# Notes:
# - Coordinates are pixel indices in the original image (360x240).
# - We'll keep boxes in torchvision format: [x1, y1, x2, y2] (float32)

_BOX_RE = re.compile(r"\((\d+)\s+(\d+)\s+(\d+)\s+(\d+)\)")

def parse_groundtruth_txt(gt_path: str) -> Dict[str, torch.Tensor]:
    with open(gt_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [
            ln.strip() for ln in f.readlines()
            if ln.strip() and not ln.strip().startswith("%")
        ]

    n_frames = int(lines[0])
    records = lines[1:1 + n_frames]

    out: Dict[str, torch.Tensor] = {}
    for ln in records:
        parts = ln.split()
        fname = parts[0]

        boxes = []
        for m in _BOX_RE.finditer(ln):
            x1, y1, x2, y2 = map(float, m.groups())
            # Safety: ensure proper ordering
            x1, x2 = min(x1, x2), max(x1, x2)
            y1, y2 = min(y1, y2), max(y1, y2)
```

```
            boxes.append([x1, y1, x2, y2])

        if boxes:
            out[fname] = torch.tensor(boxes, dtype=torch.float32)
        else:
            out[fname] = torch.zeros((0, 4), dtype=torch.float32)

    return out
```

In [ ]:

```
BATCH_SIZE = 4
NUM_EPOCHS = 5

class OTCBVSDataset(Dataset):

    def __init__(self, folders: list, transforms=None):
        self.transforms = transforms
        self.samples = []       # list of (Path(root), filename)
        self.annotations = {}   # str(root/filename) -> boxes tensor

        for root, file in folders:
            root = Path(root)
            ann = parse_groundtruth_txt(file)
            for fname, boxes in ann.items():
                key = str(root / fname)
                self.annotations[key] = boxes
                self.samples.append((root, fname))

    def __len__(self): return len(self.samples)

    def __getitem__(self, idx):
        root, fname = self.samples[idx]
        key = str(root / fname)
        image = TF.to_tensor(Image.open(root / fname).convert("RGB"))
        boxes = self.annotations[key]
        target = {"boxes": boxes, "labels": torch.ones(len(boxes), dtype=torch.int64)}
        if self.transforms:
            image, target = self.transforms(image, target)
        return image, target

def collate_fn(batch):
    images, targets = zip(*batch)
    return list(images), list(targets)
```

```python
# Load data from all subfolders and divide data into train/val
subfolders = ["01", "02", "03", "04", "05", "06", "07", "08", "09", "10"]
folders = [(f"data/000{i}", f"data/000{i}/groundtruth.txt") for i in subfolders]

full_dataset = OTCBVSDataset(folders=folders)

# For test run
# Change BATCH_SIZE and NUM_EPOCHS to 2 and 1...
# full_dataset = torch.utils.data.Subset(full_dataset, range(30)) # ...and uncomment this

# Using 80/20 division
n_train = int(0.8 * len(full_dataset))
n_val   = len(full_dataset) - n_train
train_dataset, val_dataset = torch.utils.data.random_split(
    full_dataset, [n_train, n_val], generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,  num_workers=0, collate_fn=collate_fn)
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False, num_workers=0, collate_fn=collate_fn)

print(f"Train: {n_train}  |  Val: {n_val}")
```

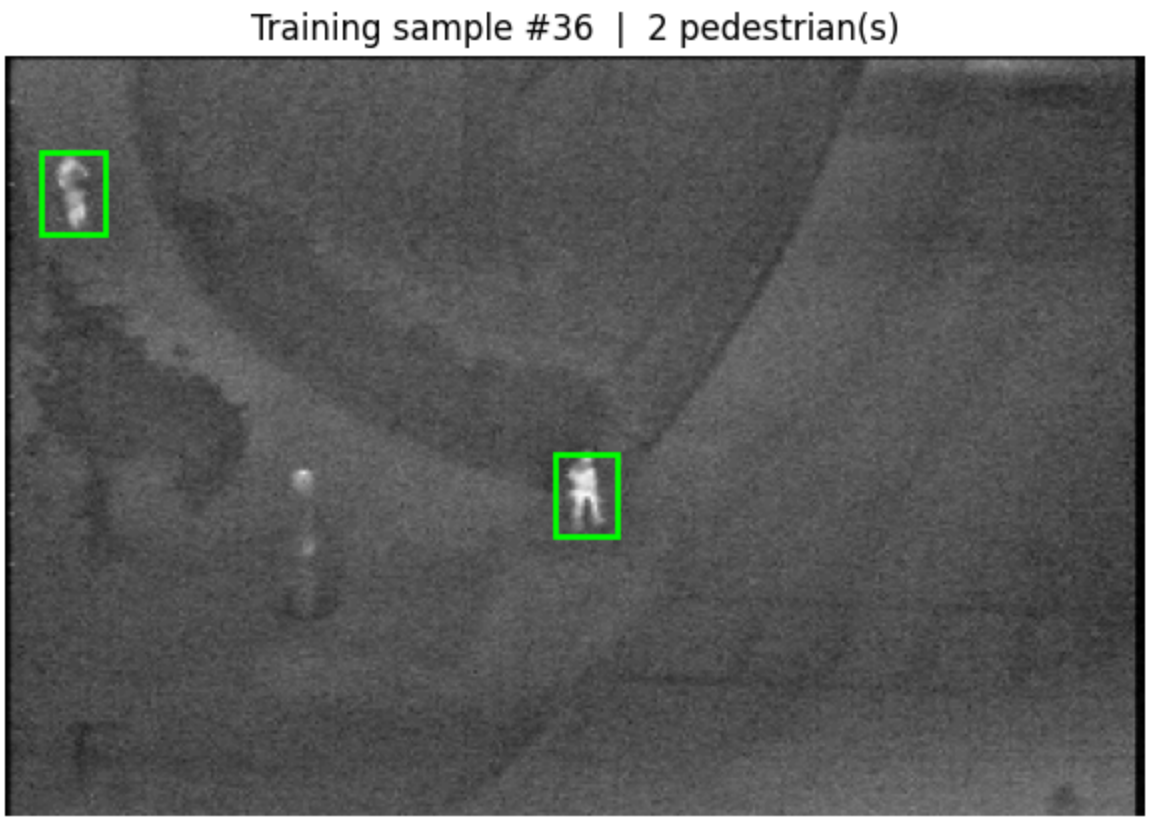
```
Train: 227  |  Val: 57
```

Visualize Two Training Samples

In [13]:

```python
def show_sample(image, target, title=""):
    fig, ax = plt.subplots(1, figsize=(6, 5))
    ax.imshow(image.permute(1,2,0).numpy(), cmap="gray")
    for box in target["boxes"]:
        x1,y1,x2,y2 = box.tolist()
        ax.add_patch(patches.Rectangle((x1,y1), x2-x1, y2-y1, lw=2, edgecolor="lime", facecolor="none"))
    ax.set_title(title); ax.axis("off")
    plt.tight_layout(); plt.show()

for i in random.sample(range(len(train_dataset)), 2):
    img, tgt = train_dataset[i]
    show_sample(img, tgt, f"Training sample #{i}  |  {len(tgt['boxes'])} pedestrian(s)")
```

Training sample #162 | 1 pedestrian(s)



Training & Evaluation Utilities

1. train_one_epoch

2. evaluate_map (mean Average Precision)

3. time_inference

In [ ]:
```
def train_one_epoch(model, loader, optimizer, device, epoch=None):
    """Train 1 epoch and calculate mean loss."""
    model.train()
    total_loss = 0.0
```

```python
    pbar = tqdm(loader, desc=f"Epoch {epoch}" if epoch else "Train")
    for images, targets in pbar:
        images  = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k,v in t.items()} for t in targets]
        loss = sum(model(images, targets).values())
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        total_loss += loss.item()
        pbar.set_postfix(loss=f"{loss.item():.4f}")
    mean_loss = total_loss / len(loader)
    print(f"  mean train loss: {mean_loss:.4f}")
    return mean_loss


@torch.no_grad()
def evaluate_map(model, loader, device):
    """mAP evaluation and returns a dict with keys like 'map', 'map_50'"""
    model.eval()
    metric = MeanAveragePrecision(iou_type="bbox")
    for images, targets in tqdm(loader, desc="Evaluating"):
        preds = model([img.to(device) for img in images])
        metric.update([{k: v.cpu() for k,v in p.items()} for p in preds],
                      [{k: v.cpu() for k,v in t.items()} for t in targets])
    return metric.compute()


# Time
@torch.no_grad()
def time_inference(model, dataset, device, n_images=100, warmup=10):
    """Average inference time per image (ms)."""
    model.eval()
    indices = random.sample(range(len(dataset)), min(n_images + warmup, len(dataset)))
    times = []
    for i, idx in enumerate(indices):
        img, _ = dataset[idx]
        img_batch = [img.to(device)]
        if device.type == "cuda": torch.cuda.synchronize()
        t0 = time.perf_counter()
        _ = model(img_batch)
        if device.type == "cuda": torch.cuda.synchronize()
        if i >= warmup:
            times.append((time.perf_counter() - t0) * 1000)
    return float(np.mean(times)), float(np.std(times))
```

Two-stage Detector: Faster R-CNN

1. Load pretrained Faster R-CNN (ResNet-50 FPN v2)
2. Freeze backbone
3. Replace box predictor: 1 foreground class + background (num_classes=2)
4. Optimizer: SGD lr=0.005, momentum=0.9, weight_decay=0.0005

In [ ]:
```
# Load pretrained RetinaNet
frcnn = fasterrcnn_resnet50_fpn_v2(weights=FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT)

# Freeze backbone
for p in frcnn.backbone.parameters(): p.requires_grad = False

# Replace box predictor head (num_classes = 1 foreground + 1 background)
in_features = frcnn.roi_heads.box_predictor.cls_score.in_features
frcnn.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes=2)

# Stochastic Gradient Descent with momentum and weight decay as given.
# Parameters where requires_grad=True will only be updated
frcnn = frcnn.to(device)
frcnn_optimizer = torch.optim.SGD(
    [p for p in frcnn.parameters() if p.requires_grad],
    lr=0.005, momentum=0.9, weight_decay=0.0005
)
print("Faster R-CNN ready. Trainable params:",
      sum(p.numel() for p in frcnn.parameters() if p.requires_grad))
```

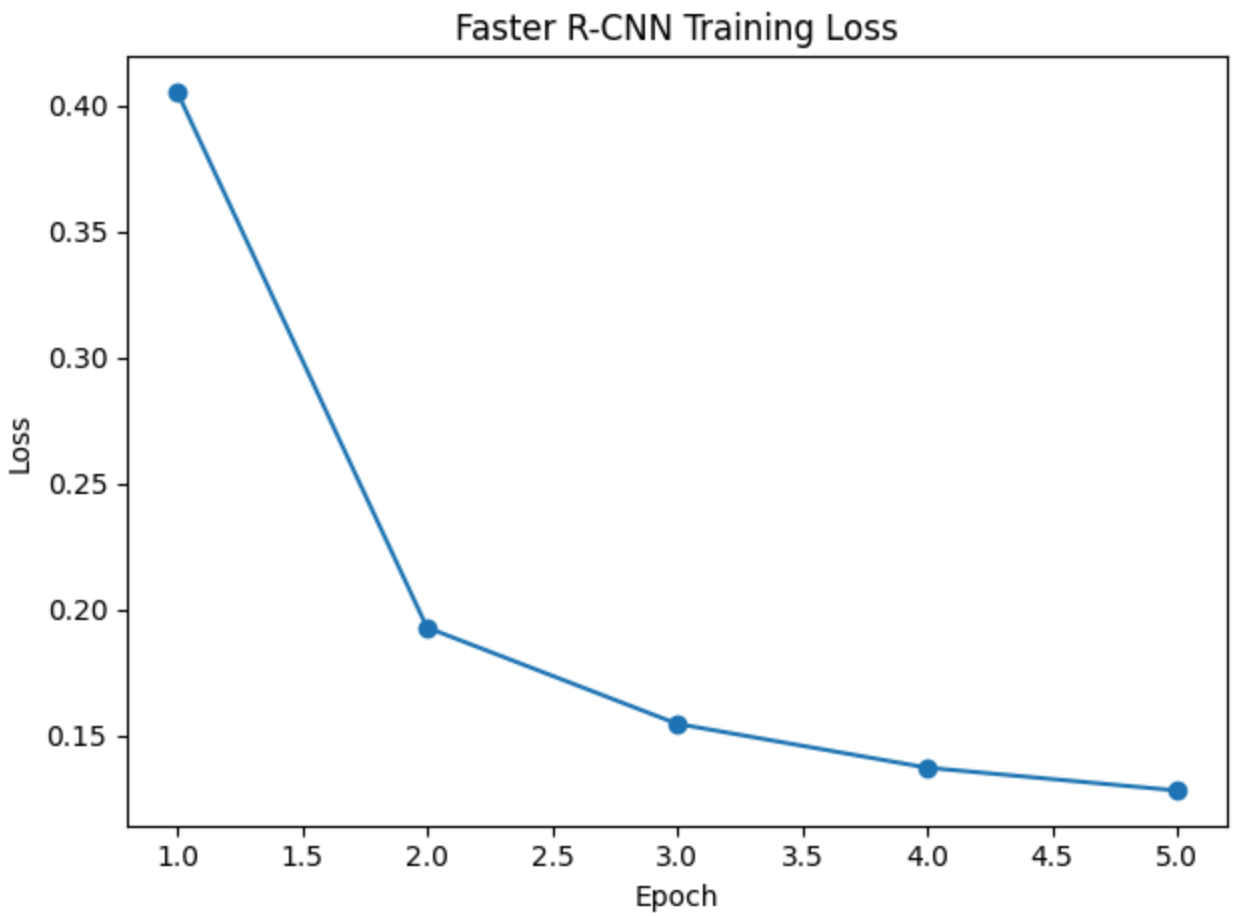
```
Faster R-CNN ready. Trainable params: 16401689
```

Train Faster R-CNN

In [ ]:
```
# Train Faster R-CNN and collect losses for plotting
frcnn_losses = [train_one_epoch(frcnn, train_loader, frcnn_optimizer, device, epoch=e)
                for e in range(1, NUM_EPOCHS + 1)]

plt.plot(range(1, NUM_EPOCHS+1), frcnn_losses, marker="o")
```

```
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Faster R-CNN Training Loss")
plt.tight_layout(); plt.show()
```

```
Epoch 1: 100%|██████████| 57/57 [00:33<00:00,  1.71it/s, loss=0.2413]
  mean train loss: 0.4054
Epoch 2: 100%|██████████| 57/57 [00:30<00:00,  1.86it/s, loss=0.1224]
  mean train loss: 0.1928
Epoch 3: 100%|██████████| 57/57 [00:30<00:00,  1.85it/s, loss=0.1404]
  mean train loss: 0.1547
Epoch 4: 100%|██████████| 57/57 [00:30<00:00,  1.86it/s, loss=0.2096]
  mean train loss: 0.1373
Epoch 5: 100%|██████████| 57/57 [00:30<00:00,  1.86it/s, loss=0.1091]
  mean train loss: 0.1282
```



One-stage Detector: RetinaNet

1. Load pretrained RetinaNet (ResNet-50 FPN v2)
2. Freeze backbone
3. Replace classification head: num_classes=2
4. Optimizer: SGD lr=0.005, momentum=0.9, weight_decay=0.0005

In [ ]:
```
# Load pretrained RetinaNet
retina = retinanet_resnet50_fpn_v2(weights=RetinaNet_ResNet50_FPN_V2_Weights.DEFAULT)
```

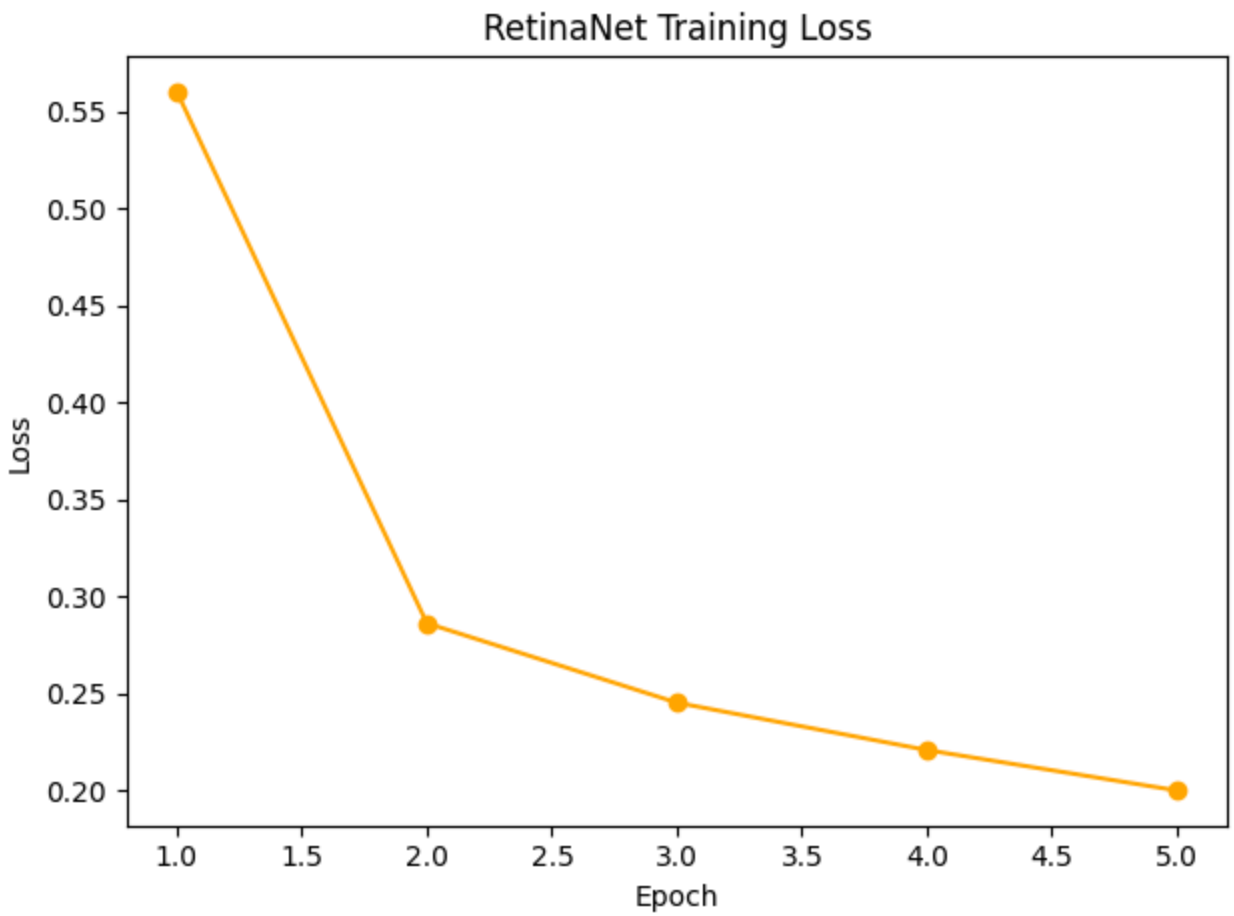
```python
# Freeze backbone
for p in retina.backbone.parameters(): p.requires_grad = False

# Replace classification head (num_classes = 1 foreground + 1 background)
num_anchors = retina.head.classification_head.num_anchors
in_channels = retina.head.classification_head.conv[0][0].in_channels
retina.head.classification_head = RetinaNetClassificationHead(
    in_channels=in_channels, num_anchors=num_anchors,
    num_classes=2, norm_layer=torch.nn.BatchNorm2d
)

# Stochastic Gradient Descent with momentum and weight decay as given and only for those with requires_grad=True
retina = retina.to(device)
retina_optimizer = torch.optim.SGD(
    [p for p in retina.parameters() if p.requires_grad],
    lr=0.005, momentum=0.9, weight_decay=0.0005
)
print("RetinaNet ready. Trainable params:",
      sum(p.numel() for p in retina.parameters() if p.requires_grad))
```

```
RetinaNet ready. Trainable params: 4847158
```

In [ ]:

```python
# Plot losses for retinanet
retina_losses = [train_one_epoch(retina, train_loader, retina_optimizer, device, epoch=e)
                 for e in range(1, NUM_EPOCHS + 1)]

plt.plot(range(1, NUM_EPOCHS+1), retina_losses, marker="o", color="orange")
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("RetinaNet Training Loss")
plt.tight_layout(); plt.show()
```

```
Epoch 1: 100%|██████████| 57/57 [00:22<00:00,  2.57it/s, loss=0.3835]
  mean train loss: 0.5601
Epoch 2: 100%|██████████| 57/57 [00:22<00:00,  2.55it/s, loss=0.2172]
  mean train loss: 0.2861
Epoch 3: 100%|██████████| 57/57 [00:22<00:00,  2.54it/s, loss=0.3084]
  mean train loss: 0.2451
Epoch 4: 100%|██████████| 57/57 [00:22<00:00,  2.53it/s, loss=0.2150]
  mean train loss: 0.2207
Epoch 5: 100%|██████████| 57/57 [00:22<00:00,  2.53it/s, loss=0.2161]
  mean train loss: 0.1998
```



Calculate val_mAP, val_mAP50, mean_infer_ms for both methods

In [19]:
```
print("Evaluating Faster R-CNN...")
frcnn_map  = evaluate_map(frcnn,  val_loader, device)

print("\nEvaluating RetinaNet...")
retina_map = evaluate_map(retina, val_loader, device)

print("\nTiming Faster R-CNN...")
frcnn_ms,  frcnn_std  = time_inference(frcnn,  val_dataset, device)

print("Timing RetinaNet...")
```

```
retina_ms, retina_std = time_inference(retina, val_dataset, device)

results = pd.DataFrame({
    "Model":      ["Faster R-CNN", "RetinaNet"],
    "val_mAP":    [round(frcnn_map["map"].item(), 4),    round(retina_map["map"].item(), 4)],
    "val_mAP50":  [round(frcnn_map["map_50"].item(), 4), round(retina_map["map_50"].item(), 4)],
    "Infer (ms)": [f"{frcnn_ms:.1f} +/- {frcnn_std:.1f}", f"{retina_ms:.1f} +/- {retina_std:.1f}"],
})
print("\n", results.to_string(index=False))
```

```
Evaluating Faster R-CNN...
Evaluating: 100%|██████████| 15/15 [00:05<00:00,  2.75it/s]
Evaluating RetinaNet...
Evaluating:   0%|          | 0/15 [00:00<?, ?it/s]c:\Users\Arde\miniconda3\envs\deep_learning_course_2026\Lib\site-packages\torchmetrics\utilities\prints.py:43: UserWarning: Encountered more than 100 detections in a single image. This means that certain detections with the lowest scores will be ignored, that may have an undesirable impact on performance. Please consider adjusting the `max_detection_threshold` to suit your use case. To disable this warning, set attribute class `warn_on_many_detections=False`, after initializing the metric.
  warnings.warn(*args, **kwargs)
Evaluating: 100%|██████████| 15/15 [00:03<00:00,  4.42it/s]
Timing Faster R-CNN...
Timing RetinaNet...

        Model  val_mAP  val_mAP50   Infer (ms)
Faster R-CNN   0.7084     0.9988 91.1 +/- 4.6
   RetinaNet   0.6372     0.9737 60.8 +/- 2.8
```

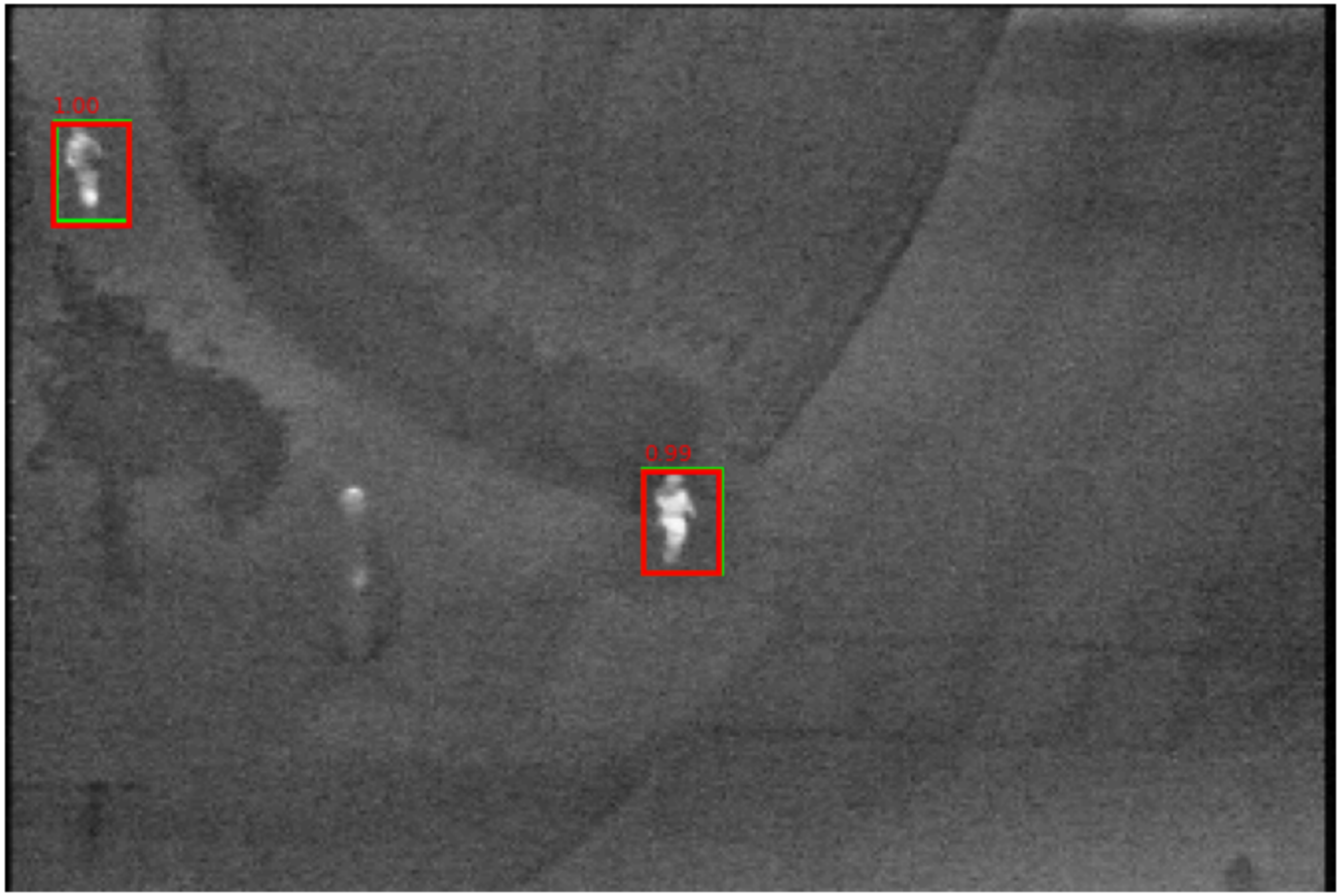
Visualize the results of two methods for couple of images from test set

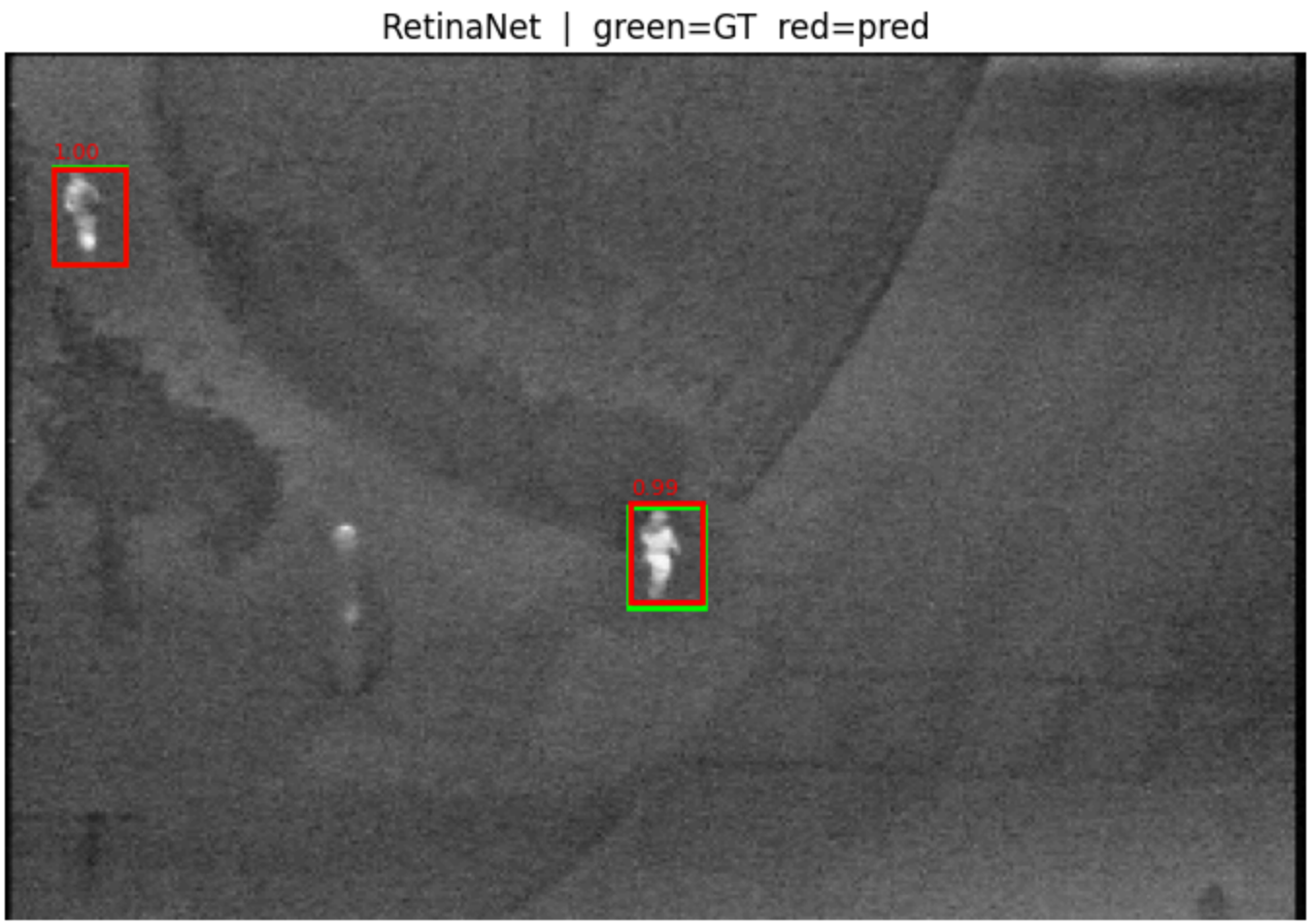
In [ ]:

```
def predict_and_show(model, dataset, device, idx, title, score_thresh=0.5):
    """Run inference on one image; green = GT, red = prediction."""
    model.eval()
    img, target = dataset[idx]

    with torch.no_grad():
        pred = model([img.to(device)])[0]

    fig, ax = plt.subplots(1, figsize=(7, 5))
    ax.imshow(img.permute(1,2,0).numpy(), cmap="gray")
```

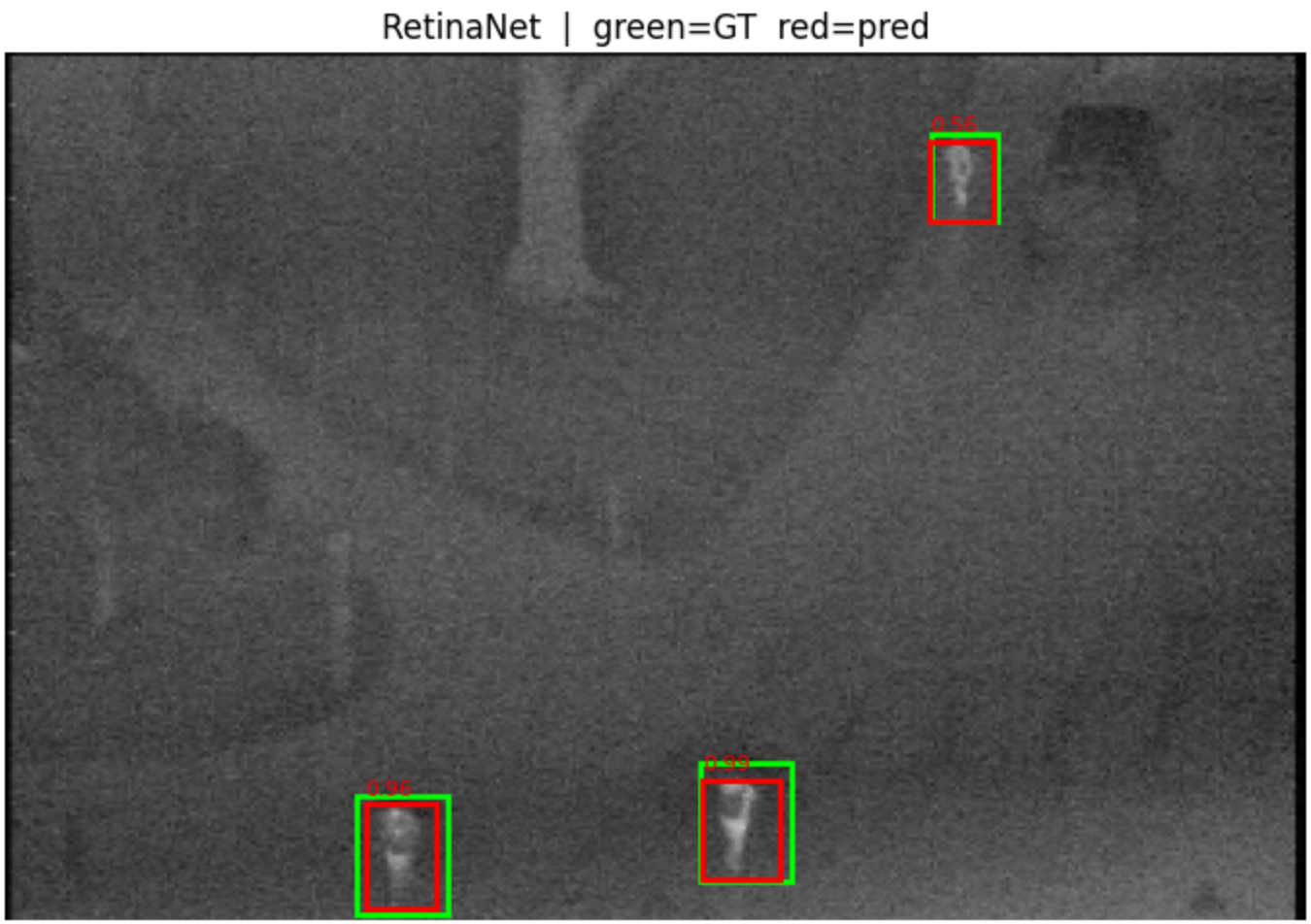
```
    # ground truth (green) for given image
    for box in target["boxes"]:
        x1,y1,x2,y2 = box.tolist()
        ax.add_patch(patches.Rectangle((x1,y1), x2-x1, y2-y1, lw=2, edgecolor="lime", facecolor="none"))

    # predictions (red) for given image
    keep = pred["scores"] >= score_thresh
    for box, score in zip(pred["boxes"][keep].cpu(), pred["scores"][keep].cpu()):
        x1,y1,x2,y2 = box.tolist()
        ax.add_patch(patches.Rectangle((x1,y1), x2-x1, y2-y1, lw=2, edgecolor="red", facecolor="none"))
        ax.text(x1, y1-3, f"{score:.2f}", color="red", fontsize=8)

    ax.set_title(f"{title}  |  green=GT  red=pred"); ax.axis("off")
    plt.tight_layout(); plt.show()

# Show predictions for 2 random images in validation set for both models
sample_indices = random.sample(range(len(val_dataset)), 2)
for idx in sample_indices:
    predict_and_show(frcnn,  val_dataset, device, idx, "Faster R-CNN")
    predict_and_show(retina, val_dataset, device, idx, "RetinaNet")
```

Faster R-CNN | green=GT red=pred




RetinaNet | green=GT red=pred
1.00
0.99

Faster R-CNN | green=GT red=pred
0.81
1.00
1.00



Conclusion

Summary

Project fine tuned two popular object detectors on the OTCBVS thermal pedestrian dataset, freezing the shared ResNet-50 FPN backbone and retraining only the detection heads.

Faster R-CNN (two-stage) first proposes candidate regions, then classifies each one. The extra region-proposal step improves local accuracy, typically gaining **higher mAP**, but at the cost of slower inference.

RetinaNet (one-stage) applies a dense anchor grid and uses focal loss to suppress the background. This should make it **faster at inference** while staying competitive in accuracy — particularly useful in real-time settings.

Key take-away: Both models benefit strongly from ImageNet pre-training on thermal images. Pretrained weights are trained with RGB data which converts well to more simple infrared images. Value of transfer learning is very good here across modalities. Training was also quite fast with dataset this small. Choose Faster R-CNN when accuracy is more important and choose RetinaNet when speed or hardware constraints matter more.